Virtual Office — Phased Technical Plan



A logic-ordered build plan: each phase depends only on the phases before it, and each ends with a verifiable exit criterion. No day numbers; schedule them to whatever time you have.

Positioning: the office that answers "what's happening?"

One-liner: Ask your office what's happening: your tasks today, where every project stands, and what's going on with every customer.

Problem: Work status is scattered. Tasks sit in a tracker, customer history in a CRM, and decisions in meeting notes nobody reopens. Answering "where are we on this?" costs a ping, a call or a meeting.

Solution: One shared memory for operations and customer relations, with the office as the interface. The AI answers from real records and cites the source, and conversations update the records without manual data entry.

Everyday question	Answered from
What is my plan for today?	Tasks due, meetings, customer follow-ups owed

Everyday question	Answered from
What is the status of Project X?	Task progress, blockers, recent decisions, owners
What is happening with Acme?	Deal stage, last contacts, open promises, risks
What changed since I was last in?	Activity across my projects and accounts
Who owns this?	Task and account ownership
What did we decide about X, and why?	Meeting records with source links

Why operations and CRM belong together: A promise made on a customer call ("we ship the integration by the 15th") becomes a task, a project milestone and a CRM note in one step. Today that is three tools and three copies of the truth. This link is the main differentiator.

Inclusion is built in, not the headline. Live captions, translated summaries and async catch-up briefings are available to everyone, which helps people across time zones and languages without singling anyone out.

Out of scope: Performance reviews, payroll, sentiment scoring or anything that monitors employees. The product gives people more context and fairer credit; it does not surveil them.

Open question: Who is the first customer: sales teams, agencies, or remote-first startups? The answer sets the demo character and which integrations matter first.

Product scope: operations and CRM in the office

The CRM is not rebuilt feature for feature. It is a CRM that fills itself from conversations, shown through three kinds of room that share one pattern: walk in, and the wall display shows live status.

Room type	Wall display shows	Typical use
Customer room	Deal stage and value, contacts, history, open commitments, risks	Calls, account reviews, pre-call briefing
Project room	Milestones, tasks, blockers, owners, recent decisions	Standups, planning, status checks

Room type	Wall display shows	Typical use
Engineering room (add-on)	Repo, open PRs, review requests, CI status	Pairing, code walkthroughs, release checks

Conventional views stay available as panels over the scene, because CRM users still expect them: a pipeline board, a table with filters, an account page with an activity timeline, and CSV import.

AI behaviour:

- Proposes CRM and task updates from transcripts as diffs; a person approves each one.
- Briefs you on entering a room: past conversations, open tasks, risks.
- Drafts follow-ups and creates tasks with owners.
- Answers "ask the office" questions with links to source records, and says "I don't have a record of that" instead of guessing.

Scope tiers

Tier	Contents
Demo	My-day panel, project status, one customer room, one project room, ask-the-office with citations, auto-capture with approval, seeded accounts and projects
Product v1	Customer lobby with guest links, CSV import, calendar sync, roles and permissions, audit log, captions and translation
Later	Email sync, HubSpot or Salesforce migration, forecasting, custom fields, proximity voice, replay of past meetings

Engineering add-on, in order of cost: GitHub status on the project wall, LiveKit screen share for pairing, then a shared snippet pad (Yjs plus CodeMirror, no code execution). A full shared IDE is out of scope; the product connects to the tools engineers already use.

0. Core Logic (what the product actually is)

The product is three loops running on one shared scene:

1. **Presence loop** — users move an avatar through a fixed isometric office; the server is the source of truth for positions.
2. **Conversation loop** — being in a room or near someone opens a voice/text channel.
3. **Memory loop** — conversations become transcripts, then structured records (summary, decisions, tasks), then searchable team memory.

The visual identity (the reference image) is the wrapper; the memory loop is the differentiator. Build in that order of *dependency*, but protect time for the memory loop because it is what makes this more than a Gather clone.

Scope rules

- Fixed orthographic camera. No orbit, no first-person.
- One office map, one avatar style, 2–4 simultaneous users.
- Server-authoritative movement. Clients send *intent*, never final position.
- No physics engine, no agent autonomy, no live transcription until the scripted path works.

1. Technologies (revised stack)

Layer	Choice	Why / change from earlier draft
App shell	React + TypeScript + Vite	Unchanged
3D scene	Three.js + React Three Fiber + Drei	Unchanged. The reference is built from clean boxes, so **procedural geometry** (box meshes, flat-shaded materials) reproduces it faster and more faithfully than hunting for GLB models
UI state	Zustand	Lightweight store shared between the canvas and React panels without re-rendering the scene

Layer	Choice	Why / change from earlier draft
Multiplayer	Colyseus	Keep. Room state sync and interpolation are built in. Alternative if deployment is painful: PartyKit / Cloudflare Durable Objects
Voice	LiveKit	Keep. Room-based first; proximity volume can be done client-side by scaling each remote track's volume by distance
Transcription	Deepgram or Whisper (via LiveKit Agents or a server worker)	Add this explicitly: the memory loop needs it. Start with a scripted transcript
Backend	Node + TypeScript (Colyseus server and a small HTTP API in the same service)	One deployable instead of two
Validation	Zod, in a shared package	One schema validates socket events, LLM output and DB inserts
Database	Supabase Postgres + Auth + pgvector	Auth added so users, workspaces and meetings have real ownership
AI	One LLM provider behind a thin interface, structured JSON output	If the hackathon requires a specific model (for example Meta's), swap it in behind the interface and check the rules early
Pathfinding	Grid A* (small custom implementation or EasyStar.js)	Needed for click-to-move and for validating server moves
Hosting	Vercel (web) + Fly.io or Railway (Colyseus; needs persistent WebSockets)	Serverless hosts cannot run the game server

Layer	Choice	Why / change from earlier draft
Testing	Vitest for movement/collision/schema logic; Playwright with two contexts for multiplayer	The two highest-risk areas get automated checks
Real-time channels	Colyseus (WebSocket) for movement, presence and chat; Supabase Realtime for record changes; LiveKit (WebRTC) for voice	Everything is live. Splitting record updates from the game server means a game-server restart does not take down status views

Decisions worth making now

- **Do not add a physics engine.** A walkability grid is enough.
- **Use instanced meshes** for repeated desks, chairs and plants to protect frame rate on average laptops.
- **Use baked-looking lighting**: one directional light, one hemisphere light, soft shadows on the floor only. Skip real-time reflections on the glass; fake it with transparency and thin frames.
- **Avatars**: stylized capsule or low-poly figures with a simple idle/walk bob are acceptable. A rigged GLB set is a stretch goal, not a dependency.

Additions for the CRM, operations and engineering-room scope

- **Records and integrations:** Postgres tables for projects, accounts and deals; GitHub REST API and webhooks (Octokit) for PR and CI status; Google Calendar sync in v1, not the demo.
- **AI actions:** LLM tool calls that return *proposed* changes. Nothing writes to a record until a person approves it, and every change is logged with its source meeting.
- **Captions and translation:** captions from the same transcription service; translated summaries through the LLM. Cheap once the transcript exists.
- **Shared snippet pad (stretch):** Yjs with CodeMirror, synced through a websocket provider on the existing server. No execution or terminal.
- **Screen share:** LiveKit's built-in screen share inside engineering rooms.

2. UI

The scene must stay the hero; panels are compact overlays.

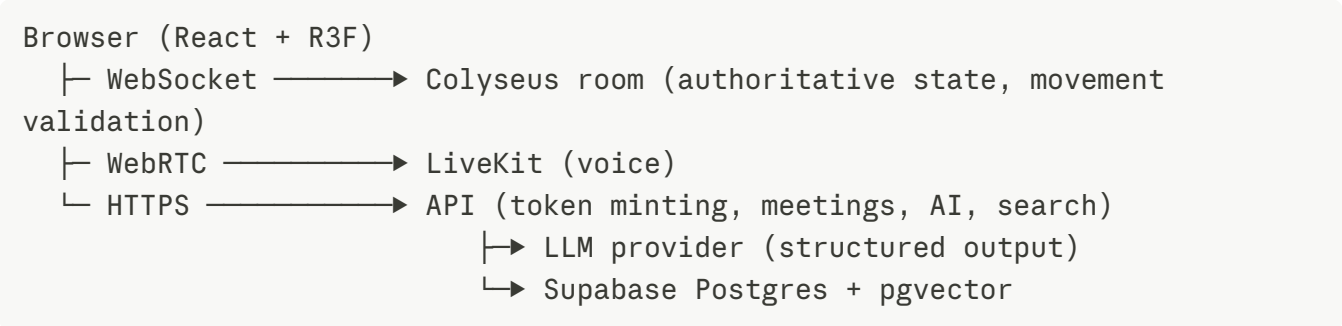
- **Canvas layer**: office scene, name labels, speaking indicators, meeting-zone highlights.
- **Left rail**: participants and presence status (available, in meeting, away).
- **Bottom bar**: mute/unmute, leave room, chat toggle.
- **Right drawer**: chat, then meeting summary and tasks, then "Ask the office" memory search. Collapsible so it never covers the scene.
- **Prompts**: a small contextual prompt when near a teammate or entering a meeting zone.
- **Failure states designed up front**: mic denied, reconnecting, LLM failed, empty memory.

Exit criterion: every panel renders with mock data before it is wired to real services.

Operations and CRM panels

- **My day:** a morning panel with tasks due, meetings and customer follow-ups in priority order.
- **Ask the office:** one chat box for status, ownership and history questions, with source links on every answer.
- **Wall displays:** live status inside customer, project and engineering rooms.
- **Approval drawer:** AI-proposed record changes shown as before-and-after diffs with approve and reject.
- **Conventional views:** pipeline board, filterable table and account timeline, opened as panels over the scene.
- **Accessibility:** captions on by default in meeting rooms, with a language selector.

3. System Architecture Design



The diagram shows three connections; a fourth carries live data. Browsers also subscribe to **Supabase Realtime** for changes to tasks, deals, milestones, proposed changes and cached PR or CI state, so wall displays and panels update without a refresh.

Real-time need	Channel	Notes
Avatar positions, presence, proximity events	Colyseus (WebSocket)	Server-authoritative, fixed tick
Text chat	Colyseus room messages	Persisted to Postgres for memory
Voice, screen share	LiveKit (WebRTC)	Room-based; token from the API
Live wall displays, my-day panel, pipeline board	Supabase Realtime	Subscribe per project, account or user; row-level security applies
Approval notifications	Supabase Realtime	New `proposed_changes` rows notify the account or project owner
Captions	LiveKit data channel or Colyseus message	Interim and final transcript segments
Shared snippet pad (stretch)	Yjs over a WebSocket provider	Same server as Colyseus
PR and CI status	GitHub webhook, API writes to Postgres, Realtime pushes to clients	Clients never poll GitHub

Rules for all channels: one reconnect strategy with backoff and state resync on reconnect; subscribe only to what the current room or view needs; keep payloads small and fetch details on demand; authorize every subscription server-side.

Shared package (`packages/shared`): world constants (grid size, zone rectangles, obstacle bounds), event types, Zod schemas. Client and server must read the *same* map definition, otherwise collision and rendering drift apart.

Data model (minimum)

- `users`, `workspaces`, `memberships`
- `meetings` (workspace, room, started/ended, transcript)
- `meeting_summaries` (summary, decisions JSON)

- `tasks` (meeting, owner, due, status)
- `meeting_chunks` (text, embedding, meeting_id) for retrieval

Added for operations and CRM

- `projects`, `milestones`: status, owner, target date.
- `accounts`, `contacts`, `deals`: stage, value, close date, owner.
- `activities`: calls, meetings, notes, linked to an account or project.
- `commitments`: promises made to customers, linked to a task and a milestone. This table joins the CRM and operations sides.
- `tasks` gain a link to either a project or an account, plus an optional PR or issue URL.
- `proposed_changes`: AI-suggested edits with target record, before and after values, source meeting, and status (pending, approved, rejected).
- `repo_links` (add-on): GitHub repo per project, cached PR and CI status.

Security basics: LiveKit tokens minted server-side and scoped to a room; Supabase row-level security by workspace; LLM keys only on the server; rate-limit movement messages.

4. Core Algorithm

The parts with real logic, each independently testable.

4.1 World model and navigation

- Define the office as a 2D walkability grid (for example 0.5 m cells) derived from furniture and wall bounds; render height is separate from walk logic.
- Obstacles are rectangles; glass walls are obstacles with door gaps.
- Click-to-move: A* over the grid, then path smoothing.

4.2 Authoritative movement

- Client sends input intent (direction or target) at a fixed rate.
- Server steps the player at a fixed tick, rejects moves into blocked cells, clamps speed, and broadcasts state.
- Client does light prediction for the local avatar and reconciles on mismatch; remote avatars are interpolated between snapshots with a small buffer (around 100 ms).

4.3 Depth and occlusion With a real 3D scene this is handled by the depth buffer. Only walls between the camera and an avatar need special treatment: fade or cut away near-side walls when someone stands behind them.

4.4 Proximity and zones

- Distance check between avatars on the server; emit `nearby` events with hysteresis (enter at 3 m, leave at 4 m) to stop flickering.
- Zones are named rectangles (meeting room A/B, kitchenette). Entering a zone joins the matching LiveKit room and marks status "in meeting".

4.5 Meeting intelligence pipeline

1. Transcript in (scripted first, live later).
2. LLM returns JSON: summary, decisions, action items with owner and due date.
3. Zod validates; on failure, retry once with the validation error, then fall back to a plain summary.
4. Persist, then chunk and embed the transcript.

4.6 Grounded recall

- Embed the question, retrieve top-k chunks, and pass only those to the LLM.
- The answer must include the source meeting; if retrieval returns nothing relevant, say so instead of guessing.

4.7 Auto-capture with approval

1. After the extraction step, map each decision, task and customer statement to a candidate record change.
2. Match entities (account, project, contact) by name and recent activity; low-confidence matches go to the person to resolve.
3. Store each change as `proposed_changes`; apply only on approval, and record who approved it.

4.8 Commitment linking

When a customer-facing meeting contains a dated promise, create one commitment and link it to a task and a milestone, so the CRM note and the project plan cannot drift apart.

4.9 My-day ranking

Score tasks by due date, deal stage or milestone impact, and whether a customer is waiting. Show the top items with the reason for each.

4.10 Status question routing (ask the office)

Classify the question (my tasks, project, account, decision history), run a structured query for status and ownership, and use semantic search only for "what did we decide" questions. Answer from retrieved records and cite them; if nothing matches, say so.

4.11 Engineering status sync (add-on)

Receive GitHub webhooks, update cached PR and CI state per project, and surface blockers (failing build, review waiting more than a set time) on the project wall.

5. Build Phases

Phase A — Visual foundation

Fixed orthographic camera at the isometric angle, floor, perimeter walls, window wall, lighting. Match the reference's framing and palette: warm oak desks, charcoal frames, light grey floor, green plants. **Exit:** a static screenshot that is recognizably the reference.

Phase B — Architecture and furniture

Glass meeting rooms with dark frames, kitchenette on the left, desk clusters with monitors and chairs, plants, shelves, printer. Build each furniture type once as a reusable component and place it from a data array (position, rotation). **Exit:** all four zones from the reference are present; the layout is defined as data in the shared package.

Phase C — Navigation

Avatar, keyboard movement, collision against the grid, name label, then click-to-move with A*. **Exit:** one user cannot walk through walls or desks; unit tests cover collision.

Phase D — Multiplayer

Colyseus room, shared player state, server-validated movement, interpolation of remote avatars, join/leave/reconnect. **Exit:** two browsers see each other move smoothly; a tampered client cannot teleport.

Phase E — Social layer

Presence, proximity detection, text chat, zone-triggered prompts, participant panel. **Exit:** two users meet at a desk or in a meeting room and exchange messages.

Phase F — Voice

LiveKit with server-issued tokens, room-based voice per meeting zone, mute control, speaking indicators, graceful mic-permission failure. **Exit:** two users talk while both avatars are visible.

Phase G — Meeting intelligence

Scripted transcript, LLM extraction, Zod validation, persistence, summary panel. **Exit:** a finished meeting produces saved summary, decisions and tasks.

Phase H — Team memory

Keyword and metadata search first, then pgvector semantic search; "Ask the office" with source citations. **Exit:** asking "what did we decide about X?" returns a grounded answer that links to the meeting.

Phase I — Operations and CRM layer

Projects, milestones, tasks, accounts, deals and commitments; the my-day panel; customer and project rooms with live wall displays; ask-the-office extended to status and ownership questions; auto-capture that produces approvable diffs; seeded demo data. **Exit:** a customer call creates a task, a milestone link and a CRM note after one approval, and a colleague's "status of Project X" answer reflects it.

Phase J — Engineering room (add-on)

GitHub status on the project wall, LiveKit screen share for pairing, tasks linked to PRs and issues. The shared snippet pad (Yjs plus CodeMirror, no execution) only if Phases I and K are stable. **Exit:** "what is blocking the release?" returns real PR and CI data, and two users can walk through code over voice and screen share.

Phase K — Deploy and harden

Deploy web and game server, configure WebSocket URLs and secrets, test on two devices, check reconnects, LLM failures and frame rate. **Exit:** a public URL that works across separate browsers with no manual steps.

Phase L — Polish and stretch

Lighting and composition polish, seeded demo workspace, better avatars, proximity voice, shared task board, door-knock flow, replay of past meetings. **Do not** start a second major subsystem here.

If time is short: Phases A–E plus a scripted version of G are the minimum that still tells the full story. Because the pitch is about daily status and customer memory, a minimal Phase I (my day, project status, one customer room) ranks above voice (F) and semantic search (H).

6. Revision (quality gates between phases)

- **After A–B:** compare side by side with the reference; fix proportions before materials.
- **After C–D:** run the two-browser Playwright test; profile frame rate with the full scene.
- **After F:** test on a different network and device, not only localhost.
- **After G–H:** test with 5 or more varied transcripts, including a short, a rambling and an empty one, and check that recall refuses to answer when nothing is stored.
- **Before demo:** record a backup video; list known limitations in the README.
- **After Phase I:** run the daily questions against seeded data, including ones with no record, and confirm no AI-proposed change is saved without approval.
- **After Phase J:** check that webhook updates reach the project wall within seconds and that screen share works across two networks.

7. Project Flow (user journey)

1. User signs in and joins the office workspace.
2. Avatar spawns in the open-plan area; teammates are visible.
3. User walks to a meeting room; the zone prompt appears and voice connects.
4. Conversation happens; presence shows "in meeting".
5. User leaves; the transcript is processed into summary, decisions and tasks.
6. Later, any teammate opens "Ask the office" and gets a cited answer from past meetings.

Demo walkthrough (a day at the office)

1. 9am: the user opens the office and asks "what's my day?" and gets three tasks and one customer follow-up.

2. They ask "status of the Acme integration?" and get progress, a blocker and the decision behind it, with a link to the meeting.
3. They walk into the Acme room, hold a call, and the call proposes a task and a CRM update; one click approves both.
4. A colleague asks the same status question and the answer has already changed.
5. In the engineering room, "what is blocking the release?" returns live PR and CI data.

8. Risks and mitigations

Risk	Mitigation
Scene never looks like the reference	Lock layout as data early; compare to the image after every visual change
Multiplayer desync	Server authority, shared map package, fixed tick
Live transcription breaks the demo	Scripted transcript path always available
LLM returns malformed output	Zod validation with one retry and a fallback
WebSocket hosting issues	Deploy a hello-world game server early, before it is needed
Frame rate on laptops	Instancing, no real-time reflections, capped pixel ratio
AI writes wrong data into CRM records	Approval step on every change, source meeting on every record, audit log
Scope creep (full CRM replacement, IDE, HR features)	Follow the scope tiers; engineering add-on only after Phase I; HR monitoring excluded
Answers are wrong when data is thin	Say "no record" instead of guessing, cite sources, seed realistic demo data
Pitch reads as three products in one	Lead with the one-liner and the daily questions; every scene element must show live data